



**POLITECNICO
DI TORINO**
Dipartimento di
Elettronica e
Telecomunicazioni



DIGITAL SYSTEMS ELECTRONICS

Academic year 2024-2025

LABORATORY NR. 3

DUE DATE: March 31, 2025

DELIVERY DATE: March 31, 2025

GROUP 20

CONTRIBUTIONS:

- Erfan Taheri
- Raman Jahandari
- Bardia Vafadar
- Ali Mansouri

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Introduction

This lab explores the design and implementation of fundamental data-path elements adders and multipliers using VHDL. We begin with a 4-bit Ripple Carry Adder (RCA) supporting signed operations and overflow detection, extend it to a 4-bit adder/subtractor, and then implement 16-bit versions of RCA, Carry-Bypass, and Carry-Select Adders. Lastly, we design a 4-bit array multiplier. Each circuit is simulated with ModelSim and deployed on the DE1 FPGA board using Quartus Prime, with performance evaluated through timing analysis.

1. 4-bit Sequential RCA (ripple carry adder)

The goal of this part is to take as inputs, two 4-bit binary operands (A) and (B) and sum them up, considering them as signed numbers in 2's complement form, giving out a 4-bit output sum (S) and the Overflow bit (OVF). In 2's complement addition, overflow occurs when the result exceeds the representable range of signed numbers. The overflow bit (OVF) is set to 1 when the carry into the sign bit differs from the carry out. For example, adding two positives gives a negative, or two negatives give a positive. This is detected by comparing the carry into and out of the sign bit:

$$\text{OVF} = \text{Carry_in_to_MSB} \oplus \text{Carry_out_of_MSB}$$

$$6 + 6 = 0110 + 0110 = 1100 (= -4)$$

Overflow occurs since both inputs are positive, but the result is negative. $\text{OVF} = 1$.

For designing this type of circuit, which is called ripple carry adder, we use four FA (full adder) components which add the bits and give out the carry outs and the output bits, three registers that are used to store the input operands (A and B) and the output sum (S). They synchronize the data with the clock signal, ensuring that inputs are loaded and the output is updated only on clock edges. This adds a latency of two clock cycles but allows stable and sequential data processing, as required in typical data-path designs. Finally, three 7-segment decoder modules to show the input numbers and the output on the board. The overflow bit is then pin assigned to red LED9 on the board to show the overflow condition. The overall circuit design is in the Figure1.

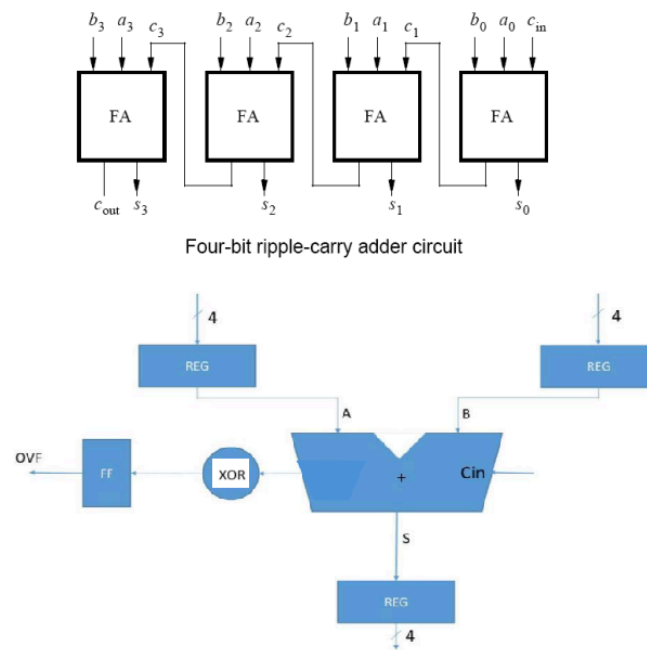


Figure 1. 4-bit signed adder with overflow and registered inputs and output, The FA's are the full adder components, The REG's are the registers and the FF is the D-flip flop.

First, starting with the registers' VHDL code, as shown in figure 2, with each rising edge of the clock, the whole input vector R is assigned into the output vector Q

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.numeric_std.all;
4
5  ENTITY regn IS
6  |   GENERIC ( N : integer := 4 );
7  |   PORT (
8  |       R      : IN  signed(N-1 DOWNTO 0);
9  |       Clock   : IN  std_logic;
10 |       Resetn  : IN  std_logic;
11 |       Q       : OUT signed(N-1 DOWNTO 0)
12 |   );
13 | END regn;
14
15 ARCHITECTURE Behavior OF regn IS
16 | BEGIN
17 |   PROCESS (Clock, Resetn)
18 |   BEGIN
19 |     IF (Resetn = '0') THEN
20 |       Q <= (OTHERS => '0');
21 |       -- RISING EDGE OF THE CLOCK
22 |     ELSIF (Clock'EVENT AND Clock = '1') THEN
23 |       Q <= R;
24 |     END IF;
25 |   END PROCESS;
26 | END Behavior;
27

```

Figure 2. The VHDL code of the Register module.

Then for the full adder component, as shown in figure 3, it has three one bit inputs, as the carry input, bit a and bit b, and as the outputs, bit s as the sum bit and c_0 as the output carrier bit.

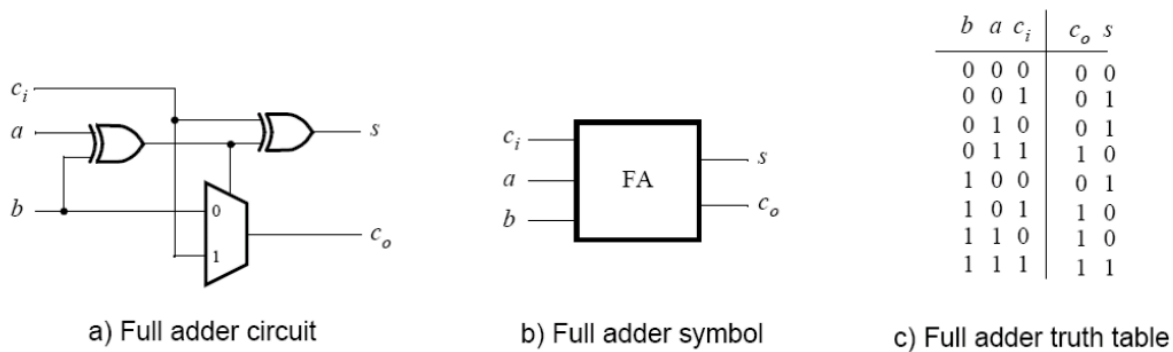


Figure 3, the circuit design of the full adder component.

The VHDL code is exactly the same as the logic circuit in the figure 3, using XOR and the multiplexer. The theory behind the outputs is in this way :

$$s = (a \text{ XOR } b) \text{ XOR } (C_i)$$

For the C_o , if the two bits a and b are different (1 and 0) then the first XOR output which is also the selection bit of the multiplexer, would be one, therefore for the C_o , we simply take the C_i bit. Otherwise, when the two bits a and b are the same, the first XOR output would be zero, taking one of the input bits (b for instance) as the C_i is the right way to assign the carry out bit. In figure 4 the VHDL code of the Full adder component is shown.

```

4 ENTITY FullAdder IS
5   PORT (
6     a      : IN  STD_LOGIC;
7     b      : IN  STD_LOGIC;
8     c      : IN  STD_LOGIC;
9     sum    : OUT STD_LOGIC;
10    carry   : OUT STD_LOGIC;
11  );
12 END FullAdder;
13
14 ARCHITECTURE behavior OF FullAdder IS
15   COMPONENT mult2to1 IS
16     PORT (
17       input  : IN  std_logic_vector(1 DOWNTO 0); -- Inputs
18       sel    : IN  STD_LOGIC;                    -- Select signal
19       output  : OUT STD_LOGIC;                   -- Output signal
20     );
21   END COMPONENT;
22
23   -- Internal signal for intermediate sum (a XOR b)
24   SIGNAL s : STD_LOGIC;
25
26 BEGIN
27   -- Calculate intermediate sum (a XOR b)
28   s <= a XOR b;
29
30   -- Calculate the final sum (s XOR c)
31   sum <= s XOR c;
32
33   -- the 2-to-1 multiplexer to calculate the carry
34   mux : mult2to1
35     PORT MAP (
36       input(0) => b,    -- Input 0 of the multiplexer is b
37       input(1) => c,    -- Input 1 of the multiplexer is c
38       sel      => s,    -- Select signal is the intermediate sum (s)
39       output   => carry -- Output is the carry bit
40     );
41
42 END behavior;

```

Figure 4. VHDL code of the FA component

Then The top level entity VHDL code that is in the Figure 5 , 6, 7 and 8, mapping the components with the internal signals in the architecture, and assigning the result of XOR operation between the last FA's carry into (c) and it's carry out (carry), to the overflow bit.

The registers input and output signals are signed vectors, however, the FA's and the seven segment decoders receive std_logic , not particularly signed vectors, therefore whenever necessary to give the vectors to the registers or to receive from the registers, in the code, we transformed the vectors from std_logic_vector to signed.

```

7  ENTITY bit_Full_Adder IS
8  PORT (
9      ain      : IN  std_logic_vector(3 DOWNTO 0); -- 4-bit input A
10     bin      : IN  std_logic_vector(3 DOWNTO 0); -- 4-bit input B
11     cin      : IN  std_logic; -- Carry input (usually '0')
12     Clock     : IN  std_logic; -- Clock signal
13     Resetn    : IN  std_logic; -- Active-low reset
14     sout      : OUT std_logic_vector(3 DOWNTO 0); -- 4-bit sum output
15     overflow  : OUT std_logic; -- Overflow flag
16     hexout, hex1 : OUT std_logic_vector(6 DOWNTO 0) -- 7-segment display outputs
17 );
18 END bit_Full_Adder;
19

```

Figure 5. The entity of the top level VHDL code, named bit_Full_Adder.

```

63  -- Internal signals.
64  -- These signals will hold the signed numbers for arithmetic.
65  SIGNAL QA_s, QB_s, QS_s : signed(3 DOWNTO 0); -- Registered signed inputs and sum
66  -- Signals to interface with the bit-level full adder chain:
67  SIGNAL QA, QB : std_logic_vector(3 DOWNTO 0); -- Conversion from signed to std_logic_vector
68  SIGNAL QS      : std_logic_vector(3 DOWNTO 0); -- Sum from the full adder chain (before re-registering)
69  SIGNAL sum_reg : std_logic_vector(3 DOWNTO 0); -- Registered sum for display
70  SIGNAL internalcarry : std_logic_vector(4 DOWNTO 0); -- Carry signals between Full Adders
71  SIGNAL internaloverflow : std_logic; -- Overflow flag
72
73  BEGIN
74  -- Convert the input operands to SIGNED and store them in registers.
75  regnA : regn
76  GENERIC MAP ( N => 4 )
77  PORT MAP (
78      R      => signed(ain), -- Convert ain to signed
79      Clock  => Clock,
80      Resetn => Resetn,
81      Q      => QA_s, -- Registered signed output for A
82  );
83
84  regnB : regn
85  GENERIC MAP ( N => 4 )
86  PORT MAP (
87      R      => signed(bin), -- Convert bin to signed
88      Clock  => Clock,
89      Resetn => Resetn,
90      Q      => QB_s, -- Registered signed output for B
91  );
92
93  -- Convert the registered signed inputs to std_logic_vector for the full adder chain.
94  QA <= std_logic_vector(QA_s);
95  QB <= std_logic_vector(QB_s);
96
97  -- Full adder chain for bitwise addition.
98  internalcarry(0) <= cin; -- Initial carry

```

Figure 6. The internal signals and the conversion of the ain and bin std_logic_vectors to signed for the registers.

```

100 FA0 : FullAdder PORT MAP (
101     a => QA(0),
102     b => QB(0),
103     c => internalcarry(0),
104     sum => QS(0),
105     carry => internalcarry(1)
106 );
107
108 FA1 : FullAdder PORT MAP (
109     a => QA(1),
110     b => QB(1),
111     c => internalcarry(1),
112     sum => QS(1),
113     carry => internalcarry(2)
114 );
115
116 FA2 : FullAdder PORT MAP (
117     a => QA(2),
118     b => QB(2),
119     c => internalcarry(2),
120     sum => QS(2),
121     carry => internalcarry(3)
122 );
123
124 FA3 : FullAdder PORT MAP (
125     a => QA(3),
126     b => QB(3),
127     c => internalcarry(3),
128     sum => QS(3),
129     carry => internalcarry(4)
130 );
131
132 -- Register the raw sum from the full adder chain.
133 regns : regn
134     GENERIC MAP ( N => 4 )
135     PORT MAP (
136         R => signed(QS), -- Convert QS to signed for registering
137         Clock => Clock,
138         Resetn => Resetn,
139         Q => QS_s -- Registered signed sum
140     );
141 -- Convert the registered signed sum back to std_logic_vector for output and display
142 sum_reg <= std_logic_vector(QS_s);
143 sout <= sum_reg;
144 -- Correct signed overflow detection: compare the carry into and out of the MSB.
145 internaloverflow <= internalcarry(4) xor internalcarry(3);
146

```

Figure 7. The Full adders mapping, and registering the output Qs, and calculating the internal overflow bit, which later using the D type flip flop will be assigned to the overflow output bit. This output bit is assigned to the red LED9 to display the overflow of the sum operation.

```

147 ff0 : flipflop PORT MAP (
148     D => internaloverflow,
149     Clock => Clock,
150     Resetn => Resetn,
151     Q => overflow
152 );
153
154 -- Instantiate the seven-segment decoders.
155 -- Note: These decoders simply display the 4-bit pattern.
156 -- For proper signed display (e.g., showing a minus sign) you'd need additional logic.
157 Decoder0 : seven_seg_dec PORT MAP (
158     s => ain,
159     hex => hex0
160 );
161
162 Decoder1 : seven_seg_dec PORT MAP (
163     s => bin,
164     hex => hex1
165 );
166
167 Decoder2 : seven_seg_dec PORT MAP (
168     s => sum_reg,
169     hex => hexout
170 );
171
172 END behavior;
173

```

Figure 8. assigning internal overflow bit to the output overflow using D type FF. and mapping the input signals ain and bin and the output sum signal sum_reg with the decoders to display them in hexadecimal format.

For the pin assignment, we used SW_3-0 for the A input, and SW_7-4 for the input B. and KEY0 as an active-low asynchronous reset input and the KEY1 as a manual clock input. The overflow bit is assigned to the red LED9 and the seven segment displays HEX0 to HEX2 are being used.

In the test bench for the clock, a signal with a period of 10 ns and a duty cycle of 50% is used, using a wait function and an infinite loop, all in one process. Then in another process, the input operands and the reset input is also changed for several times, to test the circuit. Finally the results in the modelsim waveform is shown in Figure 9. It is

shown that after two clock cycles (two rising edge of the clock cycle) , the output sum (sout) is updated with respect to the applied inputs.

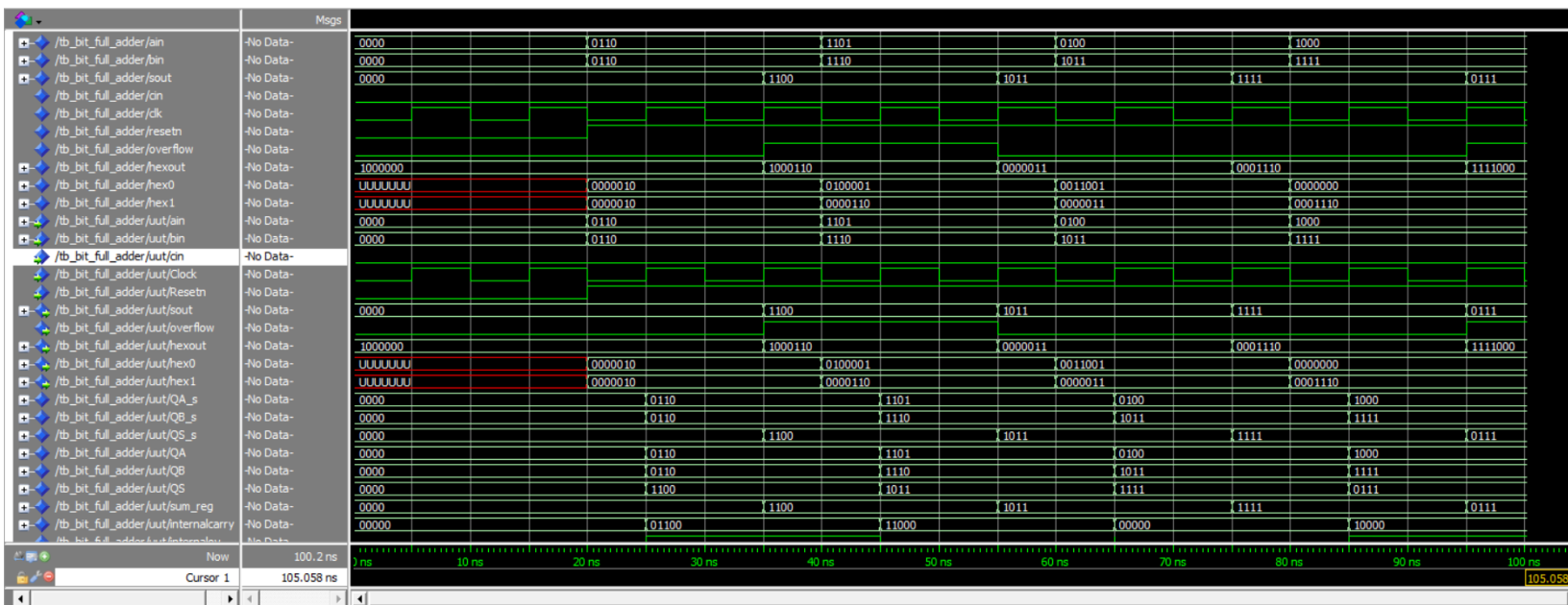


Figure 9. The waveform simulation of the circuit full adder RCA. The first 20 ns is when the reset button is triggered. That is why we do not have hex signals for the inputs, while the output hex is simply showing zero because of the initial value given to the sout which is zero. Then the reset bit is set to '1' therefore the circuit now can do the addition.

For timing simulation , first we made a .sdc file (synopsys design constraints file) that will generate the clock signal. For instance, we also used a clock signal with a period of 10 ns and duty cycle of 50%. The code is shown in figure 10.

```

1  #####
2  # Time Information
3  #####
4
5
6  set_time_format -unit ns -decimal_places 3
7
8
9  #####
10 # Create Clock
11 #####
12
13
14 create_clock -name {Clock} -period 10.000 -waveform { 0.000 5.000 }[get_ports {Clock}]

```

Figure 10. The .sdc file, generating the clock signal with a period of 10 ns and duty cycle of 50%.

Timing Analysis Results

After compiling the design in Quartus Prime and applying the clock.sdc constraint file (defining a 10 ns clock period), the TimeQuest Timing Analyzer was used to evaluate the timing performance of the 4-bit signed adder with registered inputs and outputs.

The Timing Analyzer provides maximum frequency estimates under different process-voltage-temperature (PVT) conditions. For example, under the Slow 1100 mV, 85°C model,

which represents one of the worst-case scenarios, the circuit’s maximum operating frequency is approximately 471.25 MHz. The summary of the timing analysis results is shown in figure 11.

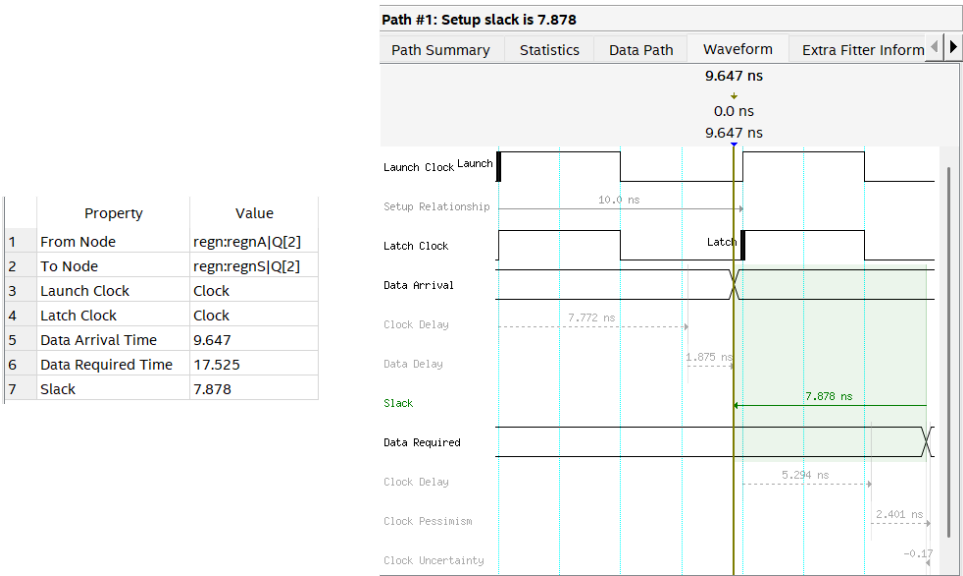


Figure 11, the summary results and the waveform of the timing analysis for 4-bit RCA.

- Clock Period Constraint: 10 ns (100 MHz)
- Maximum Operating Frequency : 471.25 MHzMHz
- Critical Path Delay (Data arrival time) : 9.647 ns = Clock delay + Data delay
- Slack: +7.878 ns

According to the TimeQuest Timing Analyzer, the critical path starts at regnA,Q[2] and ends at regnS,Q[2], passing through the bit-slice logic of the full adder. The total data arrival time for this path is 9.647 ns, while the required time is 17.525 ns, resulting in a positive setup slack of 7.878 ns. This confirms that the design meets the timing constraints with adequate margin. (The required time is not 10 ns because of clock network delay, skew, uncertainty, and Quartus timing margin)

2. 4-bit Sequential Adder/Subtractor

Building on Part 1’s design, we enhanced the 4-bit adder to support subtraction while preserving all original components: the same full adders, registers (maintaining 2-cycle latency), flip-flop for overflow, and 7-segment decoder. The only addition is an Op control signal that activates 2's complement subtraction. All other features including the overflow detection (XOR of carry bits) and DE1 board implementation (7-segment displays/LEDR9) remain identical. This modification demonstrates how to efficiently expand functionality without altering the core architecture or timing constraints.


```

BEGIN
  -- Use 2's complement logic for subtraction
  B_modified <= bin WHEN Op = '0' ELSE NOT bin;
  cin <= Op;
  internalcarry(0) <= cin;

```

Figure 12. showing the code explained above

The circuit performs subtraction using the standard 2's complement method. When in subtract mode ($Op = '1'$, where $Op = '0'$ selects addition), the system calculates $A - B$ by: first inverting all bits of B (computing the 1's complement), and second adding 1 through the initial carry-in. This converts the operation to $A + (\text{inverted } B + 1)$, where the term $(\text{inverted } B + 1)$ gives the 2's complement negative representation of B . The existing ripple carry adder then computes this equivalent addition, meaning subtraction requires only a single inverter and carry-in control while reusing all original adder components. This efficient implementation preserves the circuit's timing characteristics while providing full arithmetic capability through proper application of digital arithmetic principles. The Op signal was mapped to SW8 on the DE1 board, allowing the user to toggle between addition and subtraction operations.

The circuit performs addition when $Op = '0'$ and subtraction when $Op = '1'$, using 2's complement logic for the latter.

The testbench verifies the 4-bit adder/subtractor by simulating four operations: $7 + 2$, $7 - 2$, $9 + 8$ (expecting overflow), and $2 - 7$ (expecting a negative result). It includes proper clock generation and a reset pulse to reflect the design's registered inputs and outputs. Each input is held for enough time to observe stable results on the output.

```

50  -- Testbench for the 4-bit adder/subtractor
51  BEGIN
52    -- Apply reset pulse
53    reset <= '0';
54    WAIT FOR 20 ns;
55    reset <= '1';
56    WAIT FOR 20 ns;
57
58    -- Test Case 1: Addition 7 + 2 = 9
59    ain_sig <= "0111"; -- 7
60    bin_sig <= "0010"; -- 2
61    Op_sig <= '0'; -- addition
62    WAIT FOR 20 ns;
63
64    -- Test Case 2: Subtraction 7 - 2 = 5
65    ain_sig <= "0111"; -- 7
66    bin_sig <= "0010"; -- 2
67    Op_sig <= '1'; -- subtraction
68    WAIT FOR 20 ns;
69
70    -- Test Case 3: Addition 9 + 8 = 17 (Overflow expected)
71    ain_sig <= "1001"; -- 9
72    bin_sig <= "1000"; -- 8
73    Op_sig <= '0'; -- addition
74    WAIT FOR 20 ns;
75
76    -- Test Case 4: Subtraction 2 - 7 = -5 (2's complement)
77    ain_sig <= "0010"; -- 2
78    bin_sig <= "0111"; -- 7
79    Op_sig <= '1'; -- subtraction
80    WAIT FOR 20 ns;
81
82    WAIT;
83  END PROCESS;
84
85  END sim;

```

Figure 13 . showing the testbench for part 2

The waveform confirms correct addition, subtraction, signed output using 2's complement, and accurate overflow detection. All results appear with the expected two cycle delay due to registered inputs ,outputs.

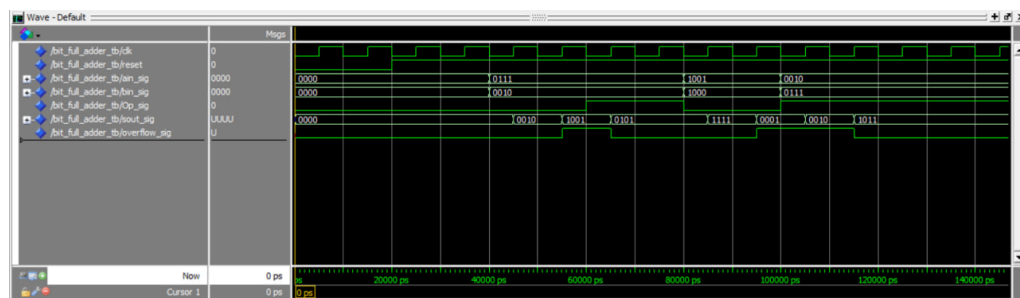


Figure 14. Simulation waveform showing part 2

Time analysis

After compiling the design and analyzing the timing with TimeQuest, the maximum operating frequency under the Slow 1100mV 85°C model is 730.99 MHz, with a restricted fmax of 717.36 MHz due to minimum period constraints. The critical path delay is approximately 4.100 ns, as reported in the Data Arrival section of the timing summary.

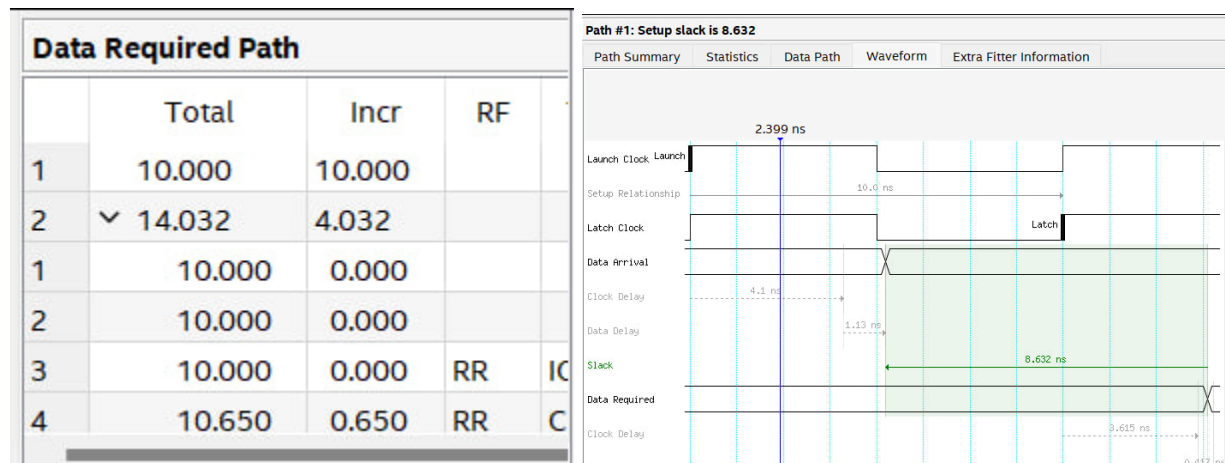


Figure 15.the summary results and the waveform of the timing analysis for 4-bit Sequential Adder/Subtractor

3. 16-bit RCA, Carry-Bypass Adder and Carry-Select Adder

a. 16-bit RCA

Here there are changes compared to the part 1 code, all in the main code, changing the N for the registers from 4 to 16, and adding a for loop to map the full adder components to the signals. And finally, to calculate the overflow, it is the same way as before, but for the sixteen FA, (C_in XOR C_out). These changes are shown in figures 15 and 16.

```
ENTITY ripple_carry_16bit IS
  PORT (
    ain      : IN  std_logic_vector(15 DOWNTO 0); -- 16-bit input A
    bin      : IN  std_logic_vector(15 DOWNTO 0); -- 16-bit input B
    cin      : IN  std_logic;                    -- Carry input
    Clock     : IN  std_logic;                    -- Clock signal
    Resetn    : IN  std_logic;                    -- Active-low reset
    sout     : OUT std_logic_vector(15 DOWNTO 0); -- 16-bit sum output
    overflow  : OUT std_logic;                    -- Overflow flag
  );
END ripple_carry_16bit;
```

(a)

<pre>regnA : regn GENERIC MAP (N => 16) PORT MAP (R => signed(ain), Clock => Clock, Resetn => Resetn, Q => QA_s); regnB : regn GENERIC MAP (N => 16) PORT MAP (R => signed(bin), Clock => Clock, Resetn => Resetn, Q => QB_s);</pre>	<pre>-- Generate full adder chain GEN_FORLOOP : FOR i IN 0 TO 15 GENERATE FAx : FullAdder PORT MAP (a => QA(i), b => QB(i), c => internalcarry(i), sum => QS(i), carry => internalcarry(i+1)); END GENERATE GEN_FORLOOP;</pre>
---	--

(b)

(c)

Figure 15. (a) the inputs ain ,bin and the output sout, all three are changed to 16 bit vectors. (b) the N is given 16, meaning 16 bit for R and Q in the regn component. (c) The for loop to map the FA's.

```
regnS : regn
  GENERIC MAP ( N => 16 )
  PORT MAP (
    R      => signed(QS),
    Clock  => Clock,
    Resetn => Resetn,
    Q      => QS_s
  );
```

(a)

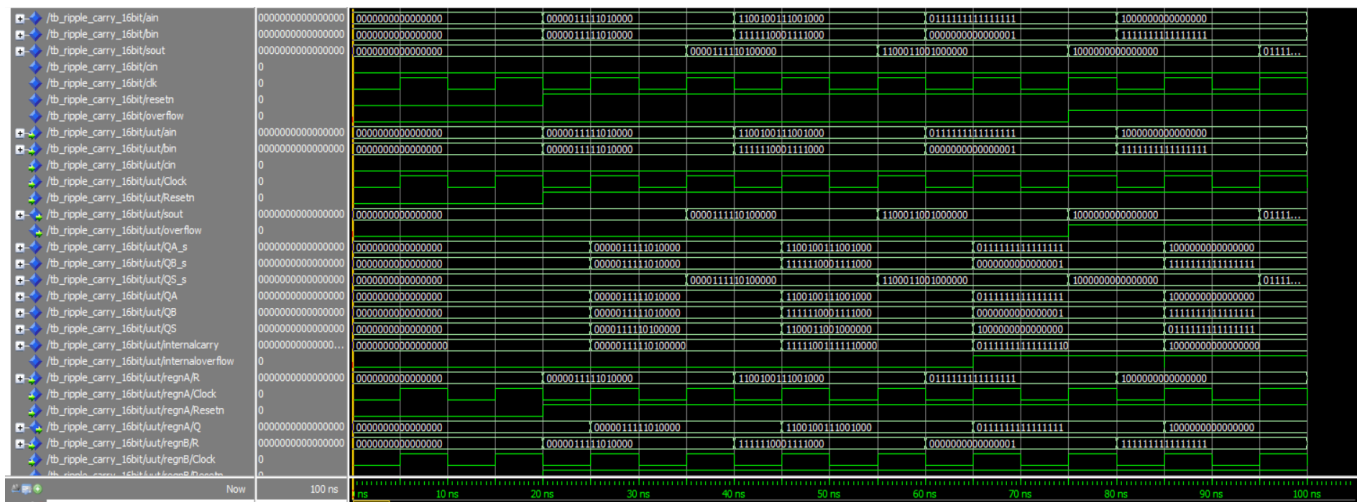
```
-- Correct signed overflow detection: compare the carry into and out of the MSB.
internaloverflow <= internalcarry(16) xor internalcarry(15);
```

(b)

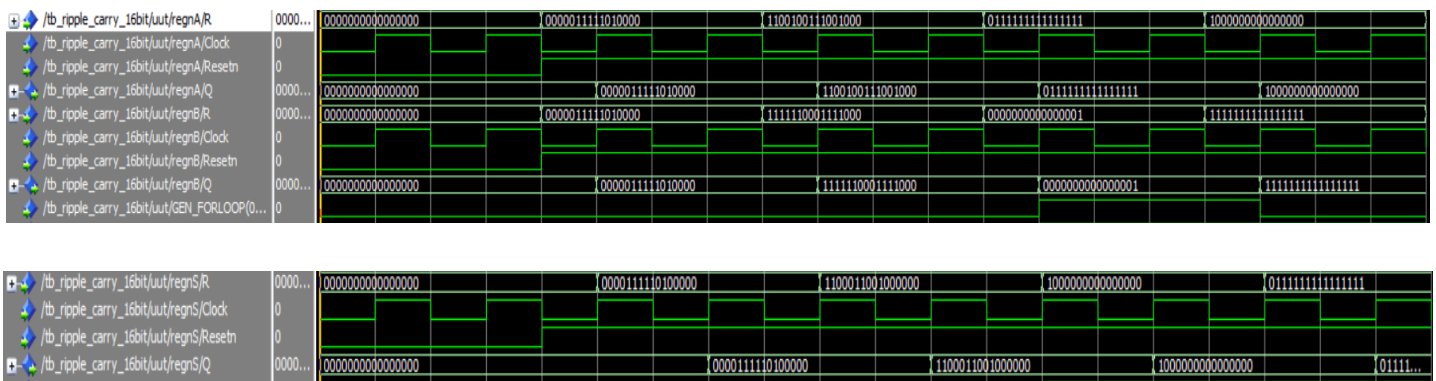
Figure 16. (a) N is assigned as 16 for the output register. (b) the internal overflow is calculated based on the last FA's input carry and output carry.

Test bench and modelsim results :

The exact same procedure as part 1, except here we have 16-bit inputs and sum output. The wait between changes is still 20 ns, so at 20 ns the reset is changed to 1 and the RCA can work from that moment. Then at 40 , 60 and 80 ns the inputs are changed, and The waveform result is shown in Figure 17 below.



Part (a) The general picture of the waveform simulation.



Part(b) The results for regnA , regnB and regnS components, and their R input and Q output updates with respect to time.

Figure 17

In figure 17 (b), it is shown that there is 5 ns delay between R update and Q update for both regnA and regnB. However, the delay is 10 ns in regnS. The reason is that the update of R in regnS happens at 25 ns, exactly at the rising edge of the clock, in this cases the register must take the before value (because of the instability at clock edge) while in regnA and B, the R signal is stable at clock edge 20 ns, therefore the Q for regnA and regnB is updated at 25 ns while the Q for regnS is updated at 35 ns. (the R in regnS is stable at 30 ns (the next rising edge after 20 ns))

Timing Analysis Results

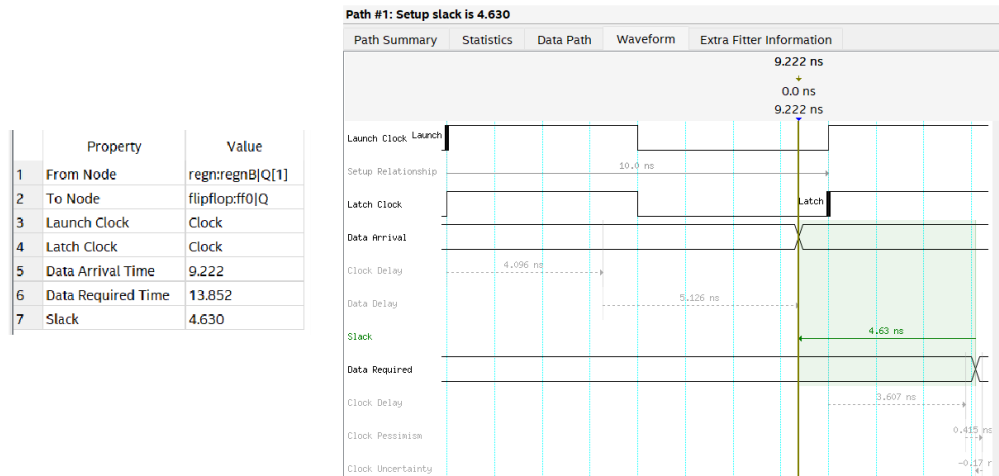


Figure 18, the summary results and the waveform of the timing analysis for 16-bit RCA.

- Clock Period Constraint: 10 ns (100 MHz)
- Maximum Operating Frequency : 186.22 MHz
- Critical Path Delay (Data arrival time) : 9.222 ns = Clock delay + Data delay
- Slack: +4.630 ns

According to the TimeQuest Timing Analyzer, the critical path starts at regnB,Q[1] and ends at flip flop Q output which is for the overflow bit, passing through the bit-slice logic of the full adder.

b. RCA with a Carry-Bypass adder

In this part, we again have four blocks, each computing 4 bit sum, however, we implemented a component called “propagate” that receives 4 bits of each input number , A and B, and evaluates the propagation condition bit, pout. This bit later is used as the selection bit for the 2to1multiplexers that will either propagate the C_in of the block or will propagate the calculated C_out of the block which is the overflow bit. Here is the schematic for the logic of this circuit. In figure 19, the whole circuit structure is drawn and the logic behind the propagate component is given.

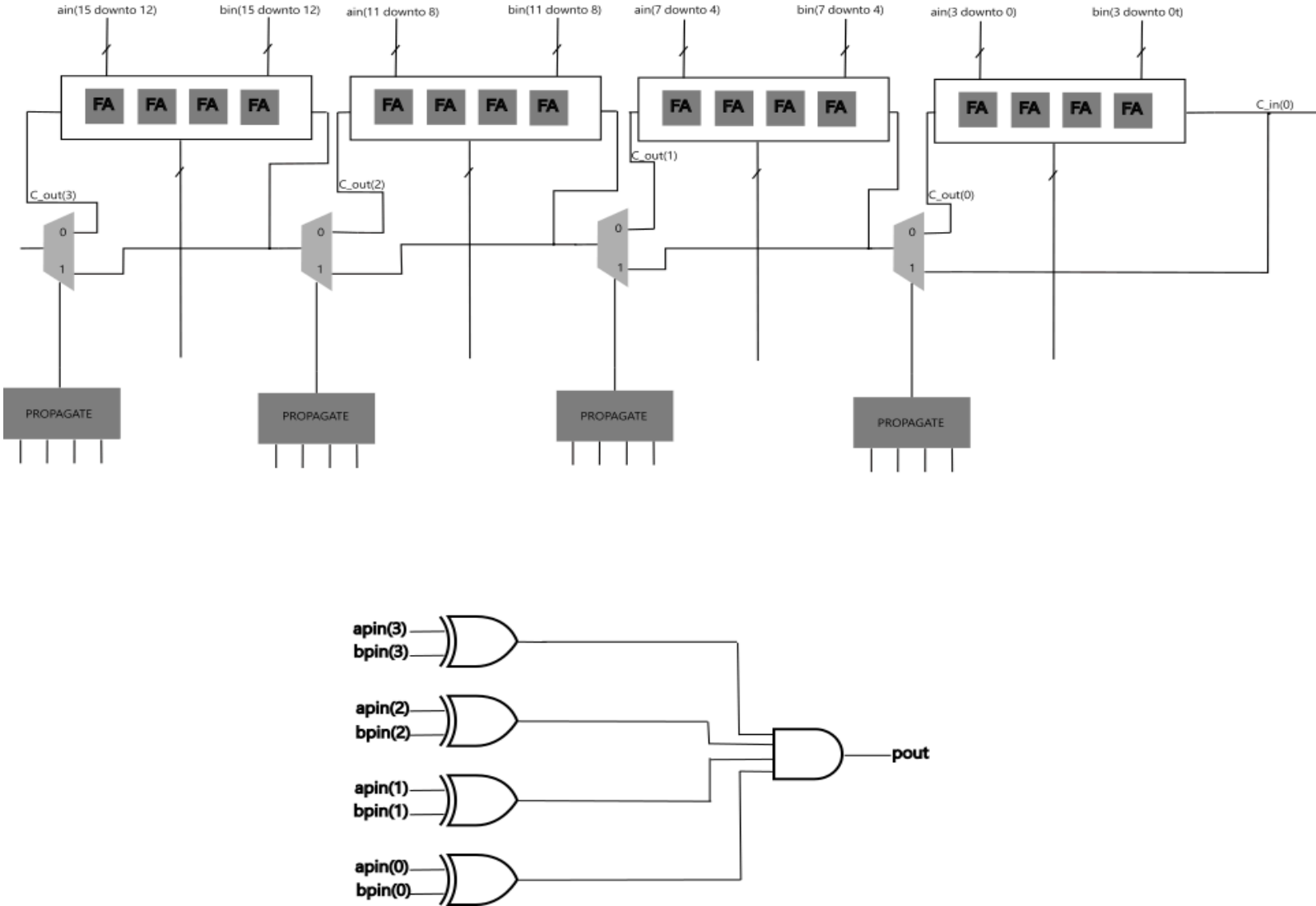


Figure 19. The whole circuit structure of the carry bypass adder. The logic of the propagate component, in general, is in this way : $a(i) \text{ XOR } b(i) = p(i)$. and the final $pout$ is the multiplication of them : $p(0) \cdot p(1) \cdot \dots \cdot p(i)$, in our case we have 4 bits therefore i is from zero to three.

In the code, the exact circuit structure is implemented, and the internal carry vector is the one that is calculated by the four blocks, and internal carry selected vector is the result of the multiplexers that are indeed going back into the blocks as carry in's. in Figure 20, a part of the top level code is shown.

```

prop0 : propagate
port map (
  apin => QA(3 downto 0),
  bpin => QB(3 downto 0),
  pout => propagation(0)
);

prop1 : propagate
port map (
  apin => QA(7 downto 4),
  bpin => QB(7 downto 4),
  pout => propagation(1)
);

prop2 : propagate
port map (
  apin => QA(11 downto 8),
  bpin => QB(11 downto 8),
  pout => propagation(2)
);

prop3 : propagate
port map (
  apin => QA(15 downto 12),
  bpin => QB(15 downto 12),
  pout => propagation(3)
);

add0 : bit_Full_Adder
port map (
  ain => QA(3 downto 0),
  bin => QB(3 downto 0),
  sout => QS(3 downto 0),
  cin => internalcarryselected(0),
  overflow => internalcarry(0)
);

add1 : bit_Full_Adder
port map (
  ain => QA(7 downto 4),
  bin => QB(7 downto 4),
  sout => QS(7 downto 4),
  cin => internalcarryselected(1),
  overflow => internalcarry(1)
);

add2 : bit_Full_Adder
port map (
  ain => QA(11 downto 8),
  bin => QB(11 downto 8),
  sout => QS(11 downto 8),
  cin => internalcarryselected(2),
  overflow => internalcarry(2)
);

add3 : bit_Full_Adder
port map (
  ain => QA(15 downto 12),
  bin => QB(15 downto 12),
  sout => QS(15 downto 12),
  cin => internalcarryselected(3),
  overflow => internalcarry(3)
);

mux0 : mult2to1
PORT MAP (
  input(0) => internalcarry(0),
  input(1) => internalcarryselected(0),
  sel    => propagation(0),
  output => internalcarryselected(1)
);

mux1 : mult2to1
PORT MAP (
  input(0) => internalcarry(1),
  input(1) => internalcarryselected(1),
  sel    => propagation(1),
  output => internalcarryselected(2)
);

mux2 : mult2to1
PORT MAP (
  input(0) => internalcarry(2),
  input(1) => internalcarryselected(2),
  sel    => propagation(2),
  output => internalcarryselected(3)
);

mux3 : mult2to1
PORT MAP (
  input(0) => internalcarry(3),
  input(1) => internalcarryselected(3),
  sel    => propagation(3),
  output => internalcarryselected(4)
);

```

(a) (b) (c)

Figure 20, (a) the propagate components portmap, (b) the four bit_full_adder blocks that compute the 4 bit summation. (c) the four multiplexers that evaluate the selected carry bits.

In simulation, it is shown in figure 21 that the circuit functions correctly via checking the internal signals such as the propagation vector which stores all the values of each propagate component. And checking the result of the summation which is the sout signal.

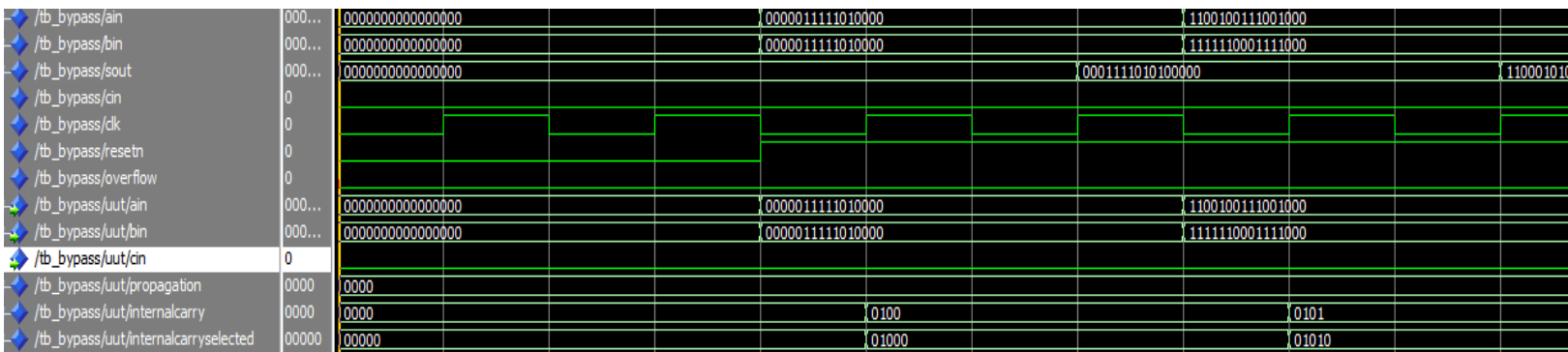
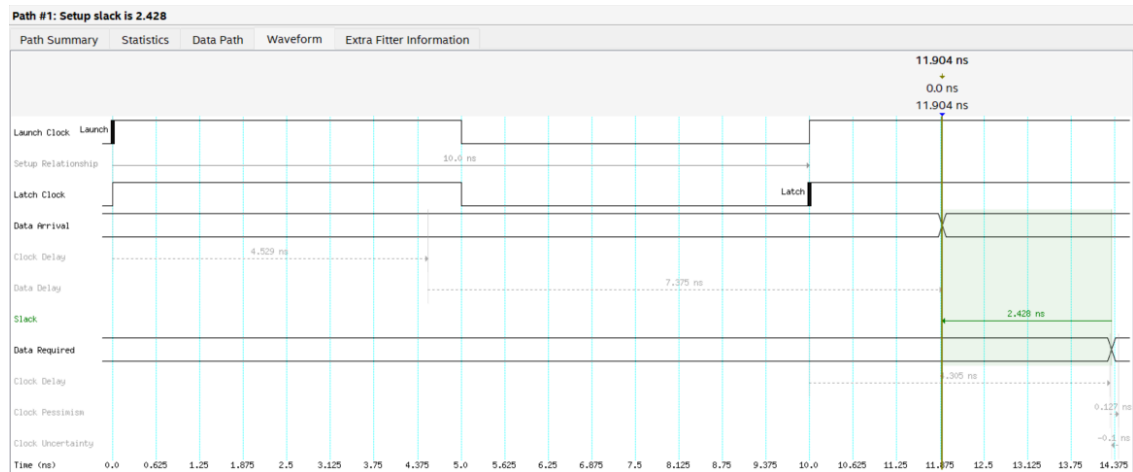


Figure 21. waveform result for the bypass adder

Timing Analysis Results



	Property	Value
1	From Node	regn:regnA[Q[2]
2	To Node	regn:regnS[Q[15]
3	Launch Clock	Clock
4	Latch Clock	Clock
5	Data Arrival Time	11.904
6	Data Required Time	14.332
7	Slack	2.428

Figure 22, the summary results and the waveform of the timing analysis for Bypass carry

- Clock Period Constraint: 10 ns (100 MHz)
- Maximum Operating Frequency : 132.08 MHz
- Critical Path Delay (Data arrival time) : 11.904 ns = Clock delay + Data delay
- Slack: +2.428 ns

According to the TimeQuest Timing Analyzer, the critical path starts at regnA,Q[2] and ends at regnS,Q[15] output.

c. RCA with a Carry-Select adder

In this method, for each four bit of the inputs, two bit_Full_Adder blocks are used, one considering a carry in equal to zero, and the other assuming carry in to be one. And based on the value of the carry_in, given by the user, the first four bit of the output is being selected, using a multiplexer, and their carry out is also selected based on the user carry_in, and that value is the selective bit for the next two blocks and so on. Here in figure 23 is the schematic for the logic behind it.

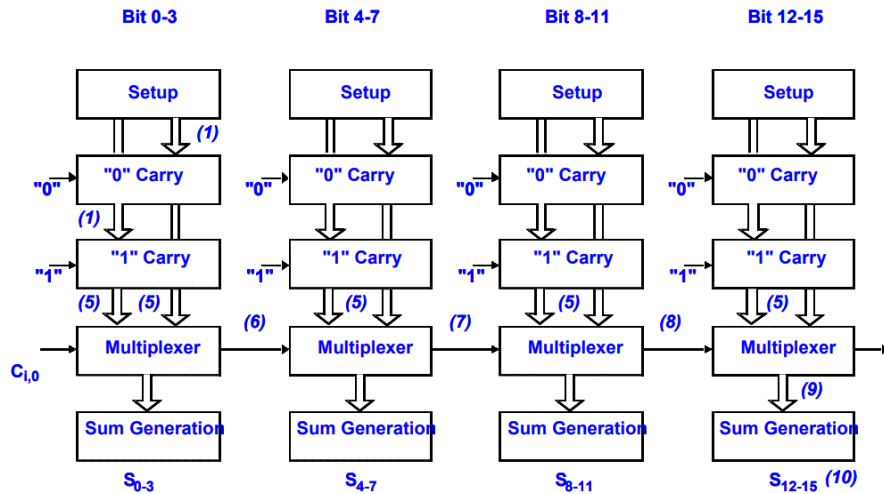


Figure 23, carry select adder

The modelsim simulation results show that the implementation functions properly, given in the figure 24.

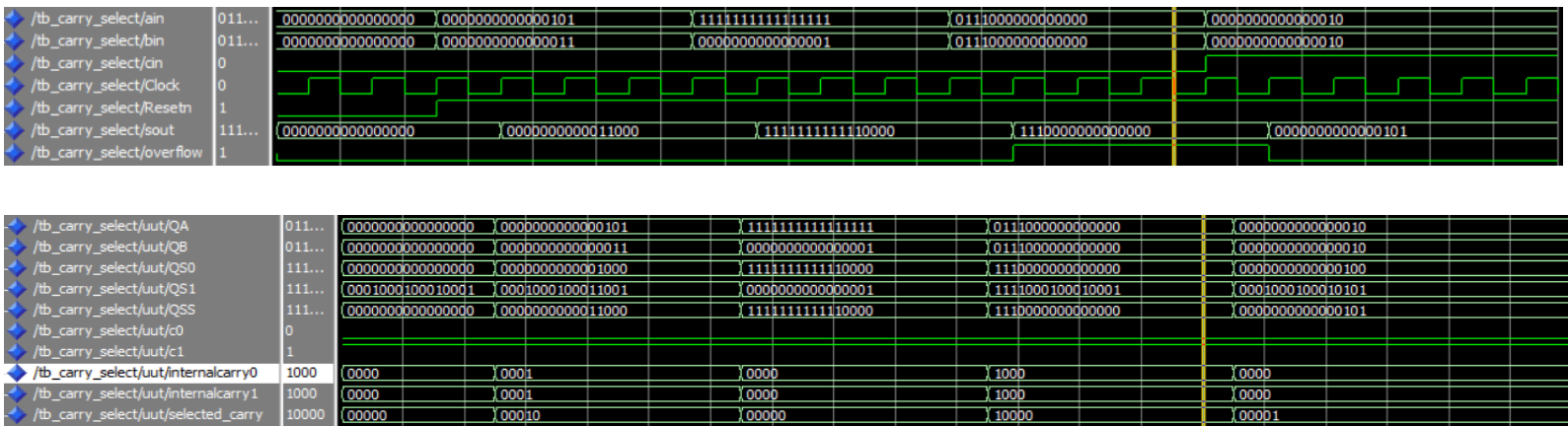


Figure 24. Five cases are shown in this waveform, first case is the reset and the others is summation, checking the overflow value and the internal signals such as QS0 that is the sum results of blocks considering carry in to be zero, and QS1, the sum results considering carry in to be one. And finally, the internalcarry0, internal carry1 and selected carry vectors that are the carry out of blocks considering c_in = 0, carry out of blocks considering c_in = 1, and the selected carries that are chosen via the multiplexer.

Timing Analysis Results

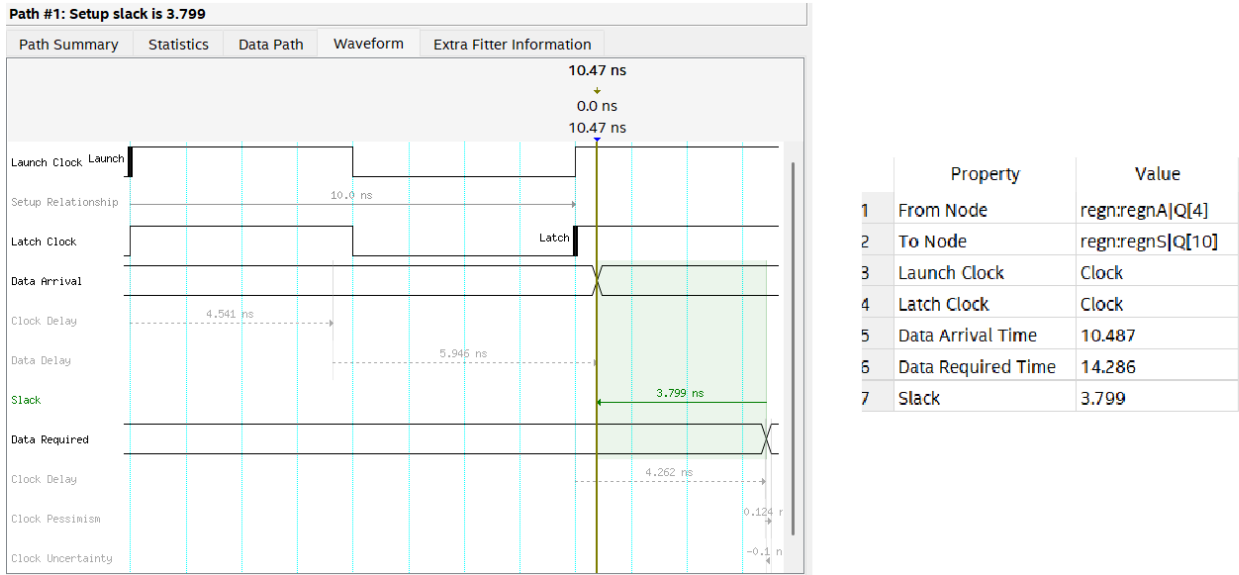


Figure25, the summary results and the waveform of the timing analysis for carry select adder.

Clock Period Constraint: 10 ns (100 MHz)

- Maximum Operating Frequency : 161.26 MHz
- Critical Path Delay (Data arrival time) : 10.487 ns = Clock delay + Data delay
- Slack: +3.799 ns

According to the TimeQuest Timing Analyzer, the critical path starts at regnA,Q[4] and ends at regnS,Q[10] output.

Comparison between three methods:

	Adder Type	Max Operating Frequency (MHz)	Critical Path Delay (ns)	Slack (ns)
1	16-bit RCA	186.22	9.222	4.63
2	Carry-Bypass Adder	132.08	11.904	2.428
3	Carry-Select Adder	161.26	10.487	3.799

Among the three adder architectures, the 16-bit Ripple-Carry Adder (RCA) achieved the highest maximum operating frequency (186.22 MHz) and the lowest critical path delay. This result, while counterintuitive, can be attributed to the RCA's simplicity and lower overhead compared to the Carry-Bypass and Carry-Select Adders at this bit-width. The Carry-Select Adder showed a good trade off between speed and complexity, while the Carry-Bypass Adder had the longest delay due to additional logic. These results are reasonable for a 16-bit design, but for wider bit-widths, more advanced adders would typically outperform RCA.

4. Multiplier

The fourth section of the laboratory focused on designing and implementing a 4-bit array multiplier capable of computing the product of two 4-bit binary numbers. The circuit was developed using a structural approach in VHDL, leveraging fundamental components such as AND gates for partial product generation and full adders for summation. The design followed the traditional paper and pencil multiplication method, where each bit of the multiplicand (A) is multiplied by each bit of the multiplier (B), resulting in 16 partial products. These partial products were then systematically summed using a network of 12 full adders, taking into account their relative bit positions to produce the final 8-bit product.

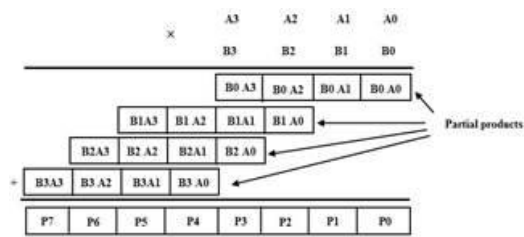


Figure 26. shows binary multiplication

This circuit was designed to receive two 4-bit binary numbers from 8 switches (SW3–0 to insert A, SW7–4 to insert B) and represent them in hexadecimal format on the 7-segment displays HEX0 and HEX1, as well as represent on HEX2 and HEX3 the product of number A and B. The 8-bit result was split into nibbles displayed on HEX2 (lower) and HEX3 (upper).

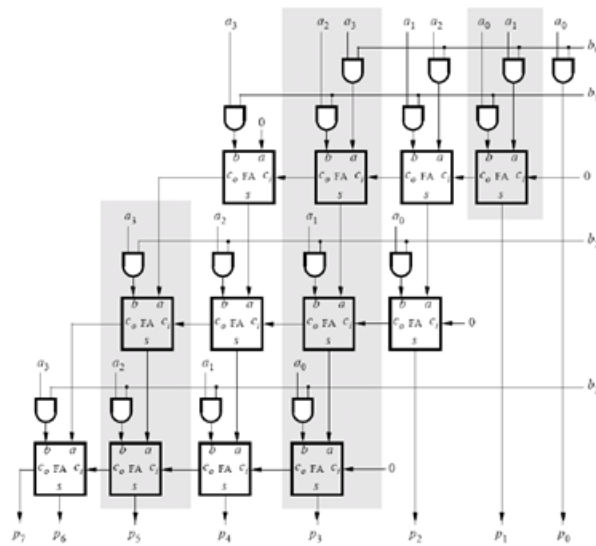


Figure 27. shows multiplication circuit

The partial products are generated using bitwise AND operations between each bit of A and B. This step replicates the rows of a paper based multiplication algorithm.

```
-- Generate partial products
p00 <= A(0) and B(0); p01 <= A(0) and B(1); p02 <= A(0) and B(2); p03 <= A(0) and B(3);
p10 <= A(1) and B(0); p11 <= A(1) and B(1); p12 <= A(1) and B(2); p13 <= A(1) and B(3);
p20 <= A(2) and B(0); p21 <= A(2) and B(1); p22 <= A(2) and B(2); p23 <= A(2) and B(3);
p30 <= A(3) and B(0); p31 <= A(3) and B(1); p32 <= A(3) and B(2); p33 <= A(3) and B(3);
```

Figure 28. shows code for partial products

The partial products are summed using a structured array of FullAdder components. Each stage handles the summation of three bits (including carries), producing both sum and carry outputs. This mimics the behavior of a hardware array multiplier, where carry values are propagated diagonally, and the results accumulate to form the full 8-bit product vector.

```
41 prod(0) <= p00;
42 FA1: FullAdder PORT MAP ( a => p01, b => p10, c => '0', sum => prod(1), carry => C1 );
43 FA2: FullAdder PORT MAP ( a => p02, b => p11, c => p20, sum => s2, carry => C2 );
44 FA3: FullAdder PORT MAP ( a => s2, b => C1, c => '0', sum => prod(2), carry => C3 );
45 FA4: FullAdder PORT MAP ( a => p03, b => p12, c => p21, sum => s4, carry => C4 );
46 FA5: FullAdder PORT MAP ( a => s4, b => p30, c => C2, sum => s5, carry => C5 );
47 FA6: FullAdder PORT MAP ( a => s5, b => C3, c => '0', sum => prod(3), carry => C6 );
48 FA7: FullAdder PORT MAP ( a => p13, b => p22, c => p31, sum => s7, carry => C7 );
49 FA8: FullAdder PORT MAP ( a => s7, b => C4, c => C5, sum => prod(4), carry => C8 );
50 FA9: FullAdder PORT MAP ( a => C6, b => C7, c => '0', sum => prod(5), carry => C9 );
51 FA10: FullAdder PORT MAP ( a => p23, b => p32, c => C8, sum => s10, carry => C10 );
52 FA11: FullAdder PORT MAP ( a => s10, b => C9, c => '0', sum => prod(6), carry => C11 );
53 FA12: FullAdder PORT MAP ( a => p33, b => C10, c => C11, sum => prod(7), carry => open );
54
55 output <= prod;
56
57
```

Figure 29. shows the code for implementing fulladder

The 8-bit product (prod) is routed to the output port and displayed as two hex digits on HEX3-HEX2, while inputs A[3:0] and B[3:0] appear on HEX0-HEX1. The 4-bit inputs (max 15×15=225) guarantee all products fit within the 8-bit output range, making overflow detection unnecessary.

KEY(1) => Clock, KEY(0) => Resetn

The circuit uses KEY1 as the manual clock input and KEY0 as the active low asynchronous reset. These control signals are used for synchronizing the operations.

The testbench applies four test cases to the 4-bit $\times$ 4-bit multiplier to verify its correctness. Inputs A and B are provided through the SW signal, and a reset followed by a clock pulse is applied using KEY. Each test case (5 $\times$ 3, 10 $\times$ 4, 15 $\times$ 15, and 7 $\times$ 8) is simulated with appropriate timing to observe stable outputs. The testbench monitors both the 8-bit result and the corresponding 7-segment outputs.

```

39      -- Test Case 1: A = 5, B = 3  $\rightarrow$  5  $\times$  3 = 15
40      SW <= "00110101"; -- B = 0011 (3), A = 0101 (5)
41      KEY <= "01"; WAIT FOR 10 ns; -- Apply reset
42      KEY <= "11"; WAIT FOR 20 ns; -- Apply clock pulse
43
44
45      -- Test Case 2: A = 10, B = 4  $\rightarrow$  10  $\times$  4 = 40
46      SW <= "01001010"; -- B = 0100 (4), A = 1010 (10)
47      KEY <= "01"; WAIT FOR 10 ns; -- Reset
48      KEY <= "11"; WAIT FOR 20 ns; -- Clock
49
50
51      -- Test Case 3: A = 15, B = 15  $\rightarrow$  15  $\times$  15 = 225
52      SW <= "11111111"; -- B = 1111 (15), A = 1111 (15)
53      KEY <= "01"; WAIT FOR 10 ns;
54      KEY <= "11"; WAIT FOR 20 ns;
55
56
57      -- Test Case 4: A = 7, B = 8  $\rightarrow$  7  $\times$  8 = 56
58      SW <= "10000111"; -- B = 1000 (8), A = 0111 (7)
59      KEY <= "01"; WAIT FOR 10 ns;
60      KEY <= "11"; WAIT FOR 20 ns;
61
62      -- End of test sequence
63      WAIT;
64  END PROCESS;
65  END behavior;

```

Figure 30. shows testbench for part 4

The waveform confirms correct operation for all test cases, showing expected 8-bit results (15, 40, 225, and 56). The red lines represent undefined values (U) during reset or before the clock pulse, which is normal with registered inputs. After the clock is applied, outputs stabilize, confirming correct synchronous behavior and reset handling.

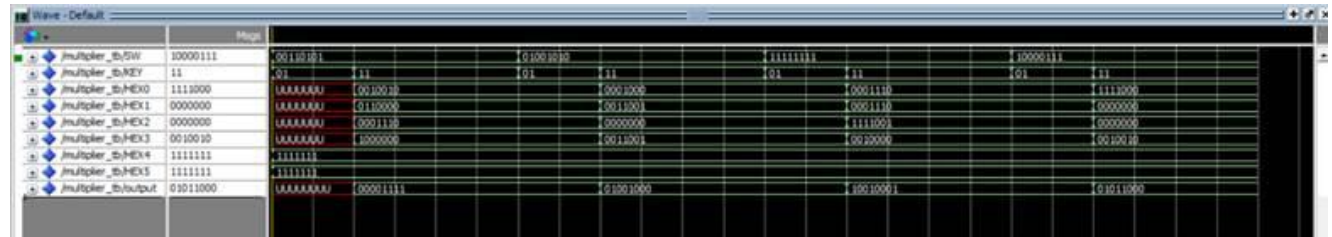


Figure 31 – Simulation waveform showing part 4

Conclusion

This lab explored the design and implementation of core data path elements, adders and multipliers, using VHDL. Starting with a 4-bit signed RCA, we extended functionality to subtraction and scaled up to 16-bit versions, comparing RCA, Carry-Bypass, and Carry-Select architectures. Surprisingly, the 16-bit RCA achieved the highest operating frequency due to its simplicity. We also implemented a 4-bit array multiplier using AND gates and full adders. All circuits were verified through simulation and timing analysis, confirming correct functionality and performance on the DE1 board.